Real-Time Self-Adaptive Agentic Intelligence for Electricity Load Forecasting Under Dynamic Demand Shifts

Sanchit Mukherjee 23BCE5126

Abstract—Short-term electricity load forecasting is normally performed with a single model that is trained offline and then deployed unchanged, so it degrades silently when consumption shifts. This report presents SAGE (Self-Adaptive aGEntic forecaster), a real-time, self-adaptive agentic system that removes this failure mode. A pool of five inexpensive forecasters (seasonal-naive, Holt–Winters, seasonal ARIMA, gradient-boosted trees and an online linear learner) is combined online by MELO, a multi-horizon aggregation method. An agentic layer monitors the residual of the served forecast with ADWIN and the Page–Hinkley test, refits the batch models immediately and at follow-up times after each alarm, and uses a contextual serving agent—a full-information contextual bandit—to re-weight the combiner and the individual models every hour. Evaluated prequentially on eight clients of the UCI ElectricityLoadDiagrams20112014 dataset, with every setting chosen on validation data, SAGE reaches 4.70% test MAPE, the primary metric, against 4.82% for MELO and 5.01% for the initial rule-based controller, and it is significantly more accurate than MELO under Diebold–Mariano tests. Under four injected demand shifts it lowers MAPE and CV-RMSE relative to MELO in every scenario and matches daily refitting with about a fifth of the refits, at roughly 31 ms per hourly step on a laptop CPU. The contribution is an integrated, reproducible adaptive forecasting system and a controlled evaluation of each of its parts, rather than a new learning algorithm.

Index Terms—short-term load forecasting, concept drift, online learning, forecast combination, contextual bandit, adaptive ensembles, agentic systems.

I. Introduction

Accurate short-term electricity load forecasting (STLF) underpins unit commitment, economic dispatch, reserve sizing and market bidding, and small percentage improvements in forecast accuracy translate into substantial operational savings. Decades of research have produced a large catalogue of STLF methods—statistical, machine-learning and deep-learning—but the dominant deployment pattern remains the same: a single model is selected and tuned on historical data, then run unchanged in production.

This pattern fails under dynamic demand shifts. Electricity consumption is non-stationary: its level, its daily and weekly shape, and its sensitivity to weather all change over time because of seasonal transitions, extreme-weather episodes, infrastructure changes and behavioural shifts. A model fitted to one regime loses accuracy when the regime changes—a phenomenon known as concept drift [7]—and, because the pipeline has no mechanism to notice or react, the loss persists until it is caught manually.

Recent research offers partial remedies. Reinforcement-learning agents can re-select or re-combine models online [2], [4]; drift detectors can flag distributional change [7], [11]; detect-then-adapt schemes retrain a model once drift is confirmed [9], [10]; online ensembles reweight their members by recent performance [14], [15], [16]; and continual-learning frameworks keep a single model current over long horizons [17], [18]. Large-language-model agents have also been proposed as forecasting controllers [5], [6]. However, these lines of work remain largely separate: the agentic controllers are select-only or rely on heavy deep reinforcement learning or on LLM inference; the drift work adapts a single model's parameters rather than managing a pool; and the online ensembles lack an explicit drift signal gating their more expensive actions.

This report specifies a system that integrates these elements into a single, deliberately lightweight controller and evaluates it under a realistic streaming protocol. Its contributions are:

- An integrated self-adaptive architecture that manages a heterogeneous pool of inexpensive forecasters—rather than adapting one network—by coupling an online combiner (MELO [16]) to an explicit statistical drift detector whose alarms trigger immediate and follow-up refits.
- A contextual serving agent, a full-information contextual bandit over the combiner and the individual models, which replaces the initial two-arm bandit controller.
- Drift detection performed on the residual of the served (deployed) forecast rather than on any individual model, with the resulting blind spot stated explicitly.
- An evaluation under a realistic streaming protocol that includes four online aggregation baselines and MELO, a daily-refitting reference, four injected demand-shift scenarios, and Diebold–Mariano significance tests.

The remainder of the report is organised as follows. Section II reviews the relevant literature and identifies the research gap, summarised in Table I. Section III presents the proposed methodology as five cooperating modules, with the system architecture in Fig. 1. Section IV reports the experimental setup, the dataset and the results, and Section V concludes.

TABLE I. Summary of the twenty most relevant works.

Ref.	Dataset	Methodology	Application	Advantages	Limitations
Cheng et al., 2026	n/a (position paper)	Conceptual framework: perception / planning / action / reflection / memory; three paradigms incl. agentic RL	Motivating and structuring agentic forecasting research	Clear taxonomy; names the agentic-RL paradigm; situates our contribution in the literature	No method, no experiments, no quantitative evidence
Feng & Zhang, 2019	Two years of load + weather data	Tabular Q-learning; ranking-based reward; moving-window selection over a 10-model pool	Short-term load forecasting model selection	Fast convergence; ~50% error reduction; low compute; simple and interpretable policy	Discrete select-only action space (no retrain/ensemble); no explicit drift signal; offline-trained policy; small tested pool
Wu, Ma & Aliev, 2026	PJM Interconnection hourly load + price, 2021-2023	MDP + Deep Q-Network; joint load/price reward; PCA-reduced state with weather and fuel-price features	Short-term load and price forecasting in energy markets	Handles coupled non-stationary targets; ~15-20% MAPE gain; interpretable policy; faster convergence than LSTM	Agent predicts values directly (no model pool); single market; no explicit drift trigger; discrete-action DQN framing
Fu, Wu & Boulet, 2022	Four real-world time-series datasets (incl. energy)	DDPG actor-critic; dilated-CNN state encoder; sMAPE + rank reward; replay buffer	Dynamic ensemble weighting for non-stationary time series	Continuous adaptive weights; fast adaptation; robust across datasets	Training instability needs exploration tricks; no retraining action; deep-RL compute; no drift-speed metrics
Das et al., 2026	Post-cutoff real-estate and equities series	LLM multi-agent decomposition (context / macro / micro / calibration) with backtesting feedback	Context-aware agentic time-series forecasting	Integrates text and events; self-calibrates on error; interpretable traces; strong accuracy	LLM inference cost and latency; no classical model pool or retraining; not tested on load; depends on frontier LLM quality
Zuo, Qin & Wang, 2025	Public load-forecasting datasets with contextual scenarios	LLM multi-agent collaboration with staged human-model interaction points	Interactive short-term load forecasting	Domain-specific; lowers technical barrier; integrates operator knowledge; accuracy gains from guidance	Human-in-the-loop (not autonomous); LLM cost; benefits depend on user insight quality; no drift/RL mechanism
Gama et al., 2014	Survey (synthetic generators + real streams incl. the Electricity benchmark)	Taxonomy of drift detection / adaptation / ensembles / windowing; prequential evaluation	Learning under concept drift (general)	Authoritative taxonomy; evaluation methodology; benchmark collection	Pre-deep-learning; classification-centric; no forecasting-specific or agentic treatment
Jagait et al., 2021	Four individual residential homes	Online Adaptive RNN + Rolling ARIMA continuously updated ensemble	Residential load forecasting under concept drift	Explicit concept-drift focus; online updating; ensemble beats constituents	Small two-model pool; fixed ensemble structure; no autonomous controller or action selection
Zhang et al., 2024	Six time-series datasets (incl. ECL / electricity)	Concept-drift detection + aggressive post-detection adaptation + Gaussian-noise data augmentation	Online time-series forecasting under accumulating concept drift	Explicit detect-then-adapt; large error reductions (33-44% MSE); model-agnostic add-on	Adaptation is retrain-only (no model switching); augmentation is a fixed heuristic; no compute-budget analysis
Zhao & Shen, 2024	Five real-world time-series datasets	Drift estimation between stale train window and current test point + adaptation generator trained on synthetic drifts	Online time-series forecasting under concept drift	Proactive (pre-ground-truth) adaptation; model-agnostic; beats reactive online learners	Not electricity-specific; no discrete model-selection or ensemble action; adaptation-generator training overhead
Lu et al., 2026	Synthetic + real image and tabular streams	Provably-better time-varying detection threshold; comparison phase bolted onto existing detectors	Adaptive concept-drift detection	Theoretical guarantee that adaptive > fixed threshold; drop-in improvement to SOTA detectors	Demonstrated on classification (not forecasting) data; adds a tuning phase; still a detector, not a full policy
Hu et al., 2026	Real-world power-system load data (extreme-weather periods)	Conditional Neural Process with similarity-based context retrieval and adaptive context reweighting	Few-shot probabilistic load forecasting under extreme regime shifts	Handles rare abrupt regimes; calibrated predictive distributions; no expensive fine-tuning	Specialised to extreme-event regime; single adaptation mechanism; not a multi-model controller
Zhu et al., 2026	Nine time-series forecasting benchmarks	MMD drift detection + dynamic expert instantiation/pruning + recurrent memory router (no test-time updates)	Non-stationary multivariate time-series forecasting	Grows/prunes the expert pool online; stable memory-based routing; SOTA accuracy; no test-time gradient	Routing is architectural (not a reward-driven policy); no retrain-vs-switch cost trade-off; not load-specific
Kolter & Maloof, 2007	STAGGER concepts, SEA concepts, CAP dataset, electricity-pricing (Elec2) dataset	Weighted online ensemble with performance-based weight decay, expert removal and expert addition	Online classification/prediction under concept drift	Adds/removes/weights experts online; fast post-drift recovery; strong empirical record	Classification-oriented; fixed hand-set rules (not learned); can grow the ensemble without bound; no compute budget
Zhang et al., 2023	Standard long-term forecasting benchmarks (incl. ECL / electricity)	Online ensemble of a time-dimension and a cross-variate model; RL-based dynamic weights over OCP	Online time-series forecasting under concept drift	>50% online error reduction; combines complementary inductive biases; faster adaptation than plain OCP	Only two base models; combination-only (no switching/retraining); RL weighting adds tuning
Wang, Goude & Yao, 2026	French national electricity load incl. COVID-19 lockdown	EWLS adaptation experts at multiple forgetting factors + parameter-free MLpol aggregation	Online load prediction under distribution shift without regime labels	Provable tracking regret; 34.7% RMSE cut; no retraining; parameter-free aggregator; load-domain demo	Linear/affine adaptation only; national-scale (not client-level); no discrete model-switch or retrain action; cost grows with number of horizons
He, Uhrweiller & Sick, 2026	Real-world power datasets (nonstationary)	Continual-learning-for-regression framework; six CL methods across three families; realistic update-policy assumptions	Long-running power forecasting under distributional drift	Self-adapts without large history storage; mitigates forgetting; practical stability/adaptation analysis	Single-model continual learning (no pool/controller); no explicit drift detector; retrain-only action space
Karaouli et al., 2025	Synthetic + real energy-forecasting benchmarks	Continual fine-tuning of TSFMs vs a specialised model; forgetting measured with/without dark experience replay	Continual time-series forecasting for energy	First systematic forgetting study for TSFMs; quantifies size vs robustness; replay closes the gap cheaply	Few model families tested; energy benchmarks limited; no online controller; fine-tuning-only adaptation
Shao et al., 2024	18 datasets incl. UCI electricity, PEMS0X, ETT, Weather	Unified BasicTS+ pipeline + heterogeneity metrics + 30+ graph/Transformer/linear models	Fair benchmarking and model-class selection for load forecasting	IEEE-reviewed; reproducible configs; quantifies when model complexity pays; includes our dataset	Point-forecast focus; heterogeneity analysis is descriptive, not an online predictor; heavy to run in full

Ref.	Dataset	Methodology	Application	Advantages	Limitations
Hong et al., 2020	Review / open energy datasets	Comprehensive review of statistical, ML, deep-learning, probabilistic and combination forecasting	Energy (demand / price / generation) forecasting	Authoritative field-level foundation; open-dataset guidance; operational-evaluation emphasis; IEEE OA	Review rather than a new method; predates the 2021+ deep-forecasting and foundation-model wave

II. Literature Review

Research relevant to a self-adaptive agentic load-forecasting system spans five areas: (A) learned and agentic control of forecasting, (B) concept-drift detection and online model adaptation, (C) online and adaptive ensembles, (D) continual learning for energy forecasting, and (E) forecasting models, benchmarks and surveys. Each is discussed below; Table I gives a structured summary of the twenty works.

A. Learned and Agentic Control of Forecasting

A growing body of work reframes forecasting from a static, single-pass mapping into a process actively managed by an agent. Cheng et al. [1] cast forecasting as an agentic loop of perception, planning, action, reflection and memory, naming agentic reinforcement learning as one way to realise it. The seminal concrete instance for load forecasting is Feng and Zhang [2], in which a tabular Q-learning agent selects, at each step, the best model from a pool of ten, reducing error by roughly 50% over any fixed model; its action space is select-only, the policy is trained offline, and no explicit drift signal is used. Fu et al. [4] (RLMC) generalise selection to continuous combination weights via a deep actor–critic. Wu, Ma and Aliev [3] cast step-by-step load and price forecasting as a Markov decision process solved with a Deep Q-Network, with the agent emitting the forecast directly. Two further works place a large language model at the centre of the loop: Nexus [5] and the interactive system of Zuo et al. [6], both of which incur LLM inference cost and latency that make a per-hour decision over long streams impractical. This area establishes the value of an agent that manages forecasting, but the available agents are either restricted to model selection [2], reliant on heavy deep reinforcement learning [3], [4], or built on LLMs [5], [6].

B. Concept-Drift Detection and Online Model Adaptation

Gama et al. [7] provide the authoritative taxonomy of concept drift and codify the prequential evaluation protocol used throughout streaming learning. In load forecasting specifically, Jagait et al. [8] combine an online adaptive recurrent network with a rolling ARIMA corrector. Zhang et al. [9] (D3A) formalise a detect-then-adapt loop; Zhao and Shen [10] (Proceed) make adaptation proactive; Lu et al. [11] prove that an adaptive detection threshold dominates any fixed one. Hu et al. [12] (AdaCNP) address extreme-weather load with a conditional neural process, and Zhu et al. [13] (Dynamic TMoE) drive a mixture-of-experts with drift detection, instantiating and pruning experts online. This area supplies mature detectors and single-model or mixture-of-experts adaptation, but the detector is seldom tied to a discrete, cost-aware controller acting over a heterogeneous pool.

C. Online and Adaptive Ensembles

Kolter and Maloof [14] (Dynamic Weighted Majority) established the template for managing a set of predictors online via performance-weighted updates and add/remove of members. Zhang et al. [15] (OneNet) apply online ensembling to forecasting, and Wang, Goude and Yao [16] (MELO) hedge adaptation experts across forgetting horizons with tracking-regret guarantees. These works show performance-weighted combination adapts well to non-stationarity, but none couples the ensemble to an explicit drift detector gating its structural actions, nor to a learned decision about whether to rely on the ensemble at all.

D. Continual Learning for Energy Forecasting

He, Uhrweiller and Sick [17] frame long-running power forecasting as continual learning for regression on a single evolving model, and Karaouli et al. [18] study catastrophic forgetting when time-series foundation models are continually fine-tuned. This area addresses sustained self-adaptation but, again, for one model rather than a managed pool.

E. Forecasting Models, Benchmarks and Surveys

Shao et al. [19] (BasicTS+) provide a reproducible benchmarking pipeline over 18 datasets and show that the best model class is dataset-dependent, and Hong et al. [20] give a field-level review of energy forecasting. Together these fix the datasets, baselines and evaluation discipline adopted here and confirm the premise underlying a model pool: no single forecaster dominates across regimes.

F. Research Gap

Across the twenty works reviewed, every ingredient of an adaptive forecasting system exists in isolation—agentic or learned control [1], [2], [4], drift detection [7], [9], [10], [11], [13], performance-driven ensembling [14], [15], [16], and continual adaptation [17], [18]—but no single work combines them. Specifically, there is no system that simultaneously (i) manages a heterogeneous pool of inexpensive models rather than adapting one network, (ii) couples an explicit statistical drift detector to a discrete, cost-aware controller action set spanning selection, blending, demotion, reinstatement and retraining, (iii) monitors the residual of the deployed (served) forecast, and (iv) is light enough to take a decision every hour over a multi-year stream without a GPU or an LLM. The proposed methodology targets this gap, and additionally asks whether a lightweight learned meta-policy measurably improves on a transparent deterministic rule set for this task.

III. Proposed Methodology

This section presents the proposed methodology: a real-time, self-adaptive agentic forecasting system for short-term electricity load prediction under dynamic demand shifts.

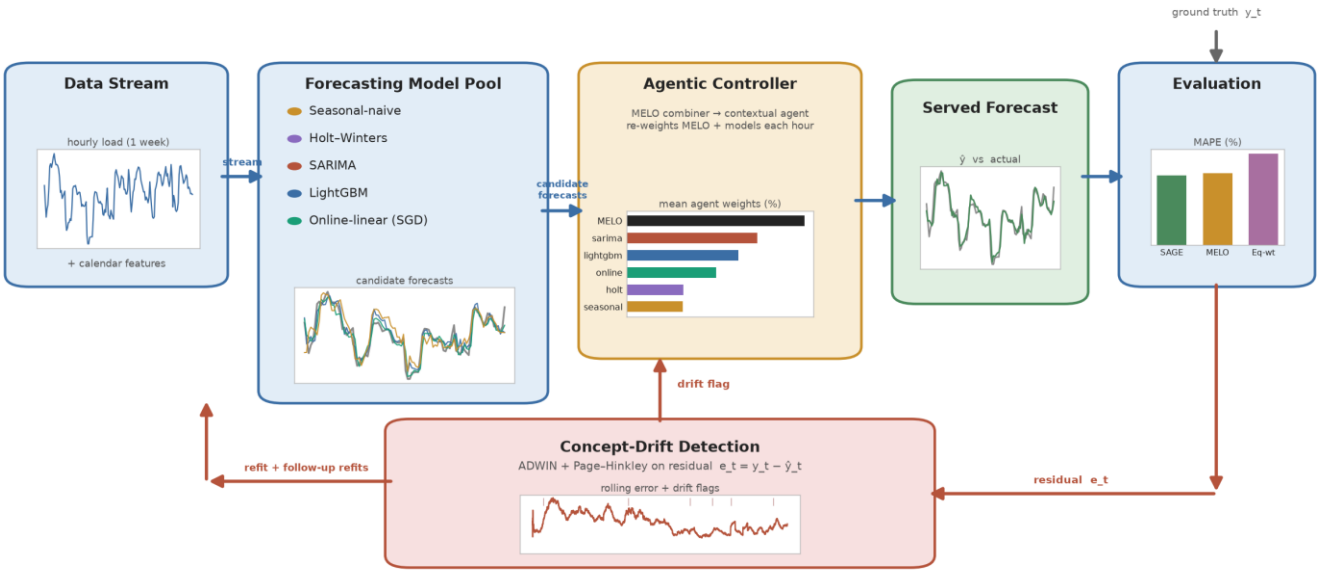


Fig. 1. Left-to-right data-flow view of the proposed system. A pool of five forecasters feeds the agentic controller, where MELO combines them and a contextual agent re-weights MELO and the individual models each hour; a demand-shift detector monitors the served forecast's error and triggers immediate and follow-up refits, and the evaluation stage returns per-candidate losses. Blue: forward forecasting path; red: drift-triggered adaptation; amber: accuracy feedback.

Conventional pipelines commit to a single model trained offline and then deployed unchanged; such a model degrades silently whenever the underlying consumption process shifts, and no corrective action is taken until the degradation is noticed manually [7]. The system proposed here removes that failure mode by maintaining a pool of lightweight, complementary forecasting models and placing an autonomous controller above the pool. At every time step the controller observes the incoming load measurement and the forecasting error it produced, tests that error stream for concept drift, and decides—from streaming feedback rather than a fixed rule—how much to trust the combined forecast versus each individual model, and when to refit.

The system is organised as a closed monitor–detect–decide–act–adapt loop of five cooperating modules. Module 1 (Data Acquisition and Preprocessing) converts the raw dataset into a clean, causally scaled, feature-augmented hourly stream. Module 2 (Forecasting Model Pool) maintains K cheap-to-retrain models that each emit a one-step-ahead forecast. Module 3 (Concept Drift Detection) applies two statistical detectors to the residual stream of the served forecast. Module 4 (Adaptive Agentic Controller) fuses the pool's forecasts with the MELO combiner, refits the batch models after a drift alarm, and re-weights the combiner and the individual models every hour through a contextual serving agent. Module 5 (Evaluation and Reward Engine) scores every served forecast, computes the accuracy, regret and adaptation-speed metrics, and returns every candidate's loss to the agent. Fig. 1 presents the design as an end-to-end data-flow diagram.

Fig. 1 traces the data flow between the modules; Fig. 2 shows the same system as layers, with the icon for each component and the retraining loop that returns from the watchdog to the model pool.

Notation and evaluation protocol. Let y_t denote the observed load at hourly step t and $\hat{y}_t$ the forecast served for that step, formed using only information available up to step $t-1$. The residual (forecast error) is $e_t = y_t - \hat{y}_t$. The pool is a set $M = \{m_1, \dots, m_K\}$ of K models, each producing its own forecast $\hat{y}_{m,t}$. The system is evaluated prequentially: each step every model predicts, the served forecast is formed, the true value is revealed and used to update models, detectors and controller, and only then does the loop advance—so no forecast uses future information [7].

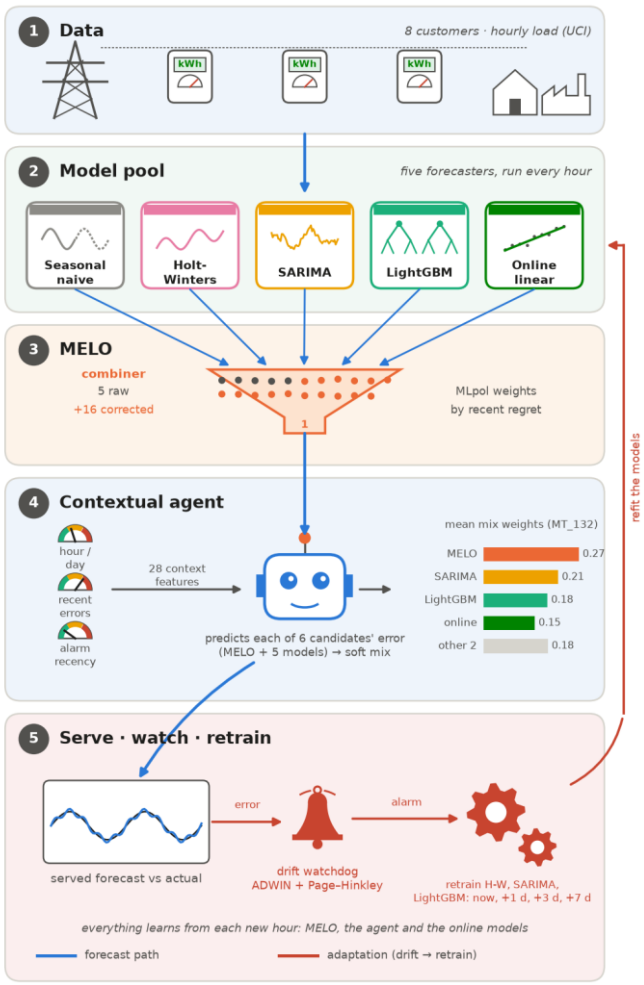


Fig. 2. Pictorial view of SAGE as five layers: hourly load from the customers' meters (1) feeds five forecasters (2), MELO combines them (3), the contextual agent mixes MELO and the five models using 28 context features (4; bars show its mean weights on MT_132), and the served forecast's error drives the drift watchdog, whose alarms retrain the batch models now and 1, 3 and 7 days later (5).

A. *Module 1: Data Acquisition and Preprocessing*

This module transforms the raw UCI ElectricityLoadDiagrams20112014 dataset—15-minute readings for several hundred clients over 2011–2014—into the streaming feed the rest of the system consumes. *Client selection:* each client is characterised by the coefficient of variation of its daily consumption and its mean load, and 6–8 clients are drawn by stratified sampling across the low-, medium- and high-variability terciles. *Temporal aggregation:* the 15-minute series is aggregated to hourly resolution,

$$x_h = \sum_{i=1}^{4} x_{15\text{min},i}, \tag{1}$$

Equation (1) gives the hourly aggregate, where $x_{15\text{min},i}$ are the four quarter-hour readings in hour h. *Feature construction:* each timestamp is annotated with hour-of-day, day-of-week, month and a holiday flag. *Causal normalisation:* each series is standardised,

$$z_t = \frac{x_t - \mu_w}{\sigma_w}, \tag{2}$$

Equation (2) standardises each series, where μ_w, σ_w are computed only from data at or before step t (refreshed every six weeks), so no future information leaks backward. *Prequential partition:* each client's timeline is split chronologically into a warm-start block (first 50%), a validation block (next 15%, used once for hyperparameter tuning), and a test block (final 35%, on which all metrics are reported).

B. *Module 2: Forecasting Model Pool*

The pool contains K methodologically diverse, cheap-to-fit models behind a uniform predict/update interface. *Seasonal naive* forecasts the value one seasonal period earlier [20],

$$\hat{y}_{t+1} = y_{t+1-s}, \quad s \in \{24,168\} \text{ hours}. \tag{3}$$

Equation (3) is the seasonal-naive forecast: it copies the value one seasonal period earlier, where y_t is the observed load, $\hat{y}_{t+1}$ the one-step-ahead forecast, and s the seasonal period in hours (s = 24 for the daily cycle, s = 168 for the weekly cycle).

Holt–Winters exponential smoothing [21] maintains a level, trend and seasonal state updated recursively,

$$l_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(l_{t-1} + b_{t-1}), \tag{4}$$

$$b_t = \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1}, \tag{5}$$

$$s_t = \gamma(y_t - l_t) + (1-\gamma)s_{t-m}, \tag{6}$$

Equations (4)–(6) are the Holt–Winters update recursions for the level l_t, the trend b_t and the seasonal component s_t, where y_t is the observed load, m the seasonal period (24 h), and α, β, γ ∈ (0,1) the smoothing constants that weight new observations against the previous state.

and forecasts by recombination,

$$\hat{y}_{t+h} = l_t + h\, b_t + s_{t+h-m}. \tag{7}$$

Equation (7) forms the h-step-ahead Holt–Winters forecast $\hat{y}_{t+h}$ by recombining the three states, where h is the forecast horizon and l_t, b_t, s_t are the level, trend and seasonal terms from (4)–(6).

Seasonal ARIMA [22] captures autocorrelation and seasonality through the backshift operator B,

$$\Phi(B)(1-B)^d \Phi_P(B^m)(1-B^m)^D y_t = \Theta(B)\,\Theta_Q(B^m)\,\varepsilon_t, \tag{8}$$

Equation (8) is the seasonal ARIMA model in backshift form, where B is the backshift operator ($B\,y_t = y_{t-1}$), Φ(B) and Θ(B) are the non-seasonal autoregressive and moving-average polynomials, $\Phi_P(B^m)$ and $\Theta_Q(B^m)$ their seasonal counterparts at period m, d and D the non-seasonal and seasonal differencing orders, ε_t the white-noise error, and y_t the observed load.

with its order fixed once at warm-start and only re-estimated thereafter. *LightGBM* gradient-boosted trees [23] on lag and calendar features minimise a regularised additive objective,

$$\mathcal{L}(\phi) = \sum_i l\,(y_i, \hat{y}_i) + \sum_k \Omega\,(f_k). \tag{9}$$

Equation (9) is the regularised objective minimised by the LightGBM gradient-boosted trees, where L(φ) is the total loss, $l(y_i, \hat{y}_i)$ the per-sample loss between target y_i and prediction $\hat{y}_i$, f_k the k-th regression tree, and $\Omega(f_k)$ a penalty on tree complexity.

An optional *incremental* learner is updated one sample at a time by stochastic gradient descent,

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta l(y_t, \hat{y}_t). \quad (10)$$

Equation (10) is the stochastic-gradient update of the incremental learner, where θ_t are the model parameters at step t, η the learning rate, and $\nabla_\theta l(y_t, \hat{y}_t)$ the gradient of the loss with respect to the parameters.

C. Module 3: Concept Drift Detection

Two complementary detectors run on the residual stream e_t of the *served* forecast —not any individual model—because it is the deployed accuracy that must be monitored. *ADWIN* [24] (surveyed in [7]) partitions a variable-length window into an older and a newer sub-window and declares change when their means differ by more than a Hoeffding bound,

$$\varepsilon_{\text{cut}} = \sqrt{1/2m \ln 4/\delta}, \quad (11)$$

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| \geq \varepsilon_{\text{cut}} \Rightarrow \text{drift flagged.} \quad (12)$$

Equations (11)–(12) define the ADWIN change test on the served-forecast residual: (11) is the Hoeffding bound ε_{cut}, where m is the harmonic mean of the two sub-window lengths and $1 - \delta$ the confidence level; (12) flags drift when the mean residuals of the older sub-window W_0 and the newer sub-window W_1 differ by at least ε_{cut}.

The *Page–Hinkley* test [25] accumulates the signed deviation from the running mean,

$$m_T = \sum_{t=1}^{T} (e_t - \bar{e}_T - \delta_{\text{PH}}), \quad (13)$$

$$PH_T = m_T - \min_{1 \leq t \leq T} m_t, \quad \text{drift if } PH_T > \lambda, \quad (14)$$

Equations (13)–(14) define the Page–Hinkley test: m_T in (13) accumulates the signed deviation of each residual e_t from the running mean of the residuals, with δ_{PH} a slack that suppresses ordinary noise; (14) flags drift when PH_T, the rise of m_T above its historical minimum, exceeds the sensitivity threshold λ.

and responds faster than ADWIN to slow, gradual drift.

Calibration. Before entering the detectors the scale-normalised residual $|e_t|/s_t$ is clipped at 10, so that a single outage hour cannot raise an alarm on its own, and the Page–Hinkley test is one-sided, because a fall in error is no reason to refit. Its threshold ($\lambda = 50$, at least 24 observations after a reset) and ADWIN's confidence ($\delta = 0.002$) were chosen on the validation block; the first, uncalibrated setting fired on a fixed weekly rhythm rather than on real changes. The calibrated detectors raise about 0.2 alarms per week on the real data.

D. Module 4: Adaptive Agentic Controller

Module 4 is the decision-making core of SAGE, built in three layers: a MELO combiner [16] that fuses the pool, a drift response that decides when to refit, and a contextual serving agent that decides what to serve. Rolling performance tracking: the controller keeps an exponentially weighted moving average of each model's absolute error [14],

$$E_m(t) = \alpha |e_t| + (1 - \alpha) E_m(t-1), \quad \alpha = 1/W,\ W = 168. \quad (15)$$

Equation (15) is the exponentially weighted moving average of each model's absolute error, where $E_m(t)$ is model m's running error score, e_t its residual at step t, and $\alpha = 1/W$ the smoothing weight for a window of W = 168 hours (one week).

Forecast fusion (MELO). $K = 16$ correction experts each fit a time-varying affine combination of the pool forecasts with an intercept, $\tilde{z}_s = (\hat{y}_{1,s}, \ldots, \hat{y}_{M,s}, 1)$, by exponentially weighted least squares with forgetting factor γ_k,

$$\widehat{w}_{k,t} = \arg\min_w \sum_{s<t} \gamma_k^{t-1-s} (y_s - w^\top \tilde{z}_s)^2 \quad (16)$$

Equation (16) is solved recursively for 15 forgetting factors with memory lengths $1/(1-\gamma_k)$ from 20 to 5,000 h plus $\gamma_k = 1$, and expert k forecasts $\hat{y}_t^{(k)} = \widehat{w}_{k,t}^\top \tilde{z}_t$. The intercept lets an expert absorb a level shift without retraining. The pool forecasts and the 16 experts form a candidate set $c_{j,t}$ of 21 forecasts, mixed by MLpol [28],

$$\pi_{j,t} = \frac{[R_{j,t-1}]_+}{\sum_i [R_{i,t-1}]_+}, \quad \hat{y}_t^c = \sum_j \pi_{j,t} c_{j,t} \quad (17)$$

where $[\cdot]_+$ is the positive part and each candidate's regret accumulates with the squared-loss gradient,

$$R_{j,t} = R_{j,t-1} + 2(\hat{y}_t^c - y_t)(\hat{y}_t^c - c_{j,t}) \quad (18)$$

MLpol has no learning rate. MELO's only setting, its covariance-inflation scale, was chosen on the validation block, and the expert covariances are re-symmetrised and trace-capped for numerical stability over more than 9,000 hourly steps.

Drift response. On a drift flag the retrainable models (Holt–Winters, SARIMA, LightGBM) are refit at once on their usual windows, and follow-up refits are scheduled 24, 72 and 168 h later, so that each refit sees more of the new regime; the same models are also refit on a fixed 168-h schedule. This response was chosen on validation data with injected shifts over two alternatives that lost: refitting on post-change data only (one week is too little for SARIMA and LightGBM) and resetting MELO's regrets and covariances after an alarm.

Contextual serving agent. On top, an agent decides each hour how much to trust the combiner versus each individual model. Its candidates are $a_{0,t} = \hat{y}_t^c$ and $a_{m,t} = \hat{y}_{m,t}$, each scored by a scale-free loss,

$$\ell_{j,t} = \min(|y_t - a_{j,t}|/s_t,\ 10) \quad (19)$$

where s_t is a running (EWMA) scale of the combiner's absolute error. Every candidate is computed every hour, so all losses are observed after each step (full-information feedback) and no exploration is needed. Each candidate's next loss is predicted from a context vector x_t of 28 features—hour of day, weekend and holiday flags, candidate disagreement, fast and slow EWMA losses, deviations from the combiner, recent level change, and time since the last alarm and refit—by recursive least squares with forgetting factor λ,

$$\theta_j \leftarrow \theta_j + g_t(\ell_{j,t} - \theta_j^\top x_t), \quad g_t = \frac{P x_t}{\lambda + x_t^\top P x_t} \quad (20)$$

with $P \leftarrow (P - g_t x_t^\top P)/\lambda$ shared by all candidates and its trace capped. The served forecast mixes the candidates by a softmin of predicted loss with temperature τ,

$$\omega_{j,t} = \frac{\exp(-\theta_j^\top x_t/\tau)}{\sum_i \exp(-\theta_i^\top x_t/\tau)}, \quad \hat{y}_t = \sum_j \omega_{j,t} a_{j,t} \quad (21)$$

Validation selected $\lambda = 0.999$, $\tau = 0.3$, soft mixing rather than hard selection, and absolute rather than squared loss. Because each decision affects only the current hour and its outcome is observed at once, a full-information contextual

bandit is the appropriate tool rather than full reinforcement learning. The agent replaces the two-arm Discounted-UCB1 bandit of the initial design [26], [27], which only chose between the blend and the single best model. Algorithms 1 and 2 specify the control loop and the agent.

Algorithm 1. Per-step control loop of the final system

1. each model m predicts $\hat{y}_{m,t}$
2. MELO forms the experts and the combined forecast $\hat{y}_t^c$ by (16)–(17)
3. agent builds the context x_t and serves $\hat{y}_t$ by (21)
4. observe y_t; $e_t = y_t - \hat{y}_t$
5. update the models, MELO's regrets (18), the agent (19)–(20) and the scores (15)
6. feed the clipped $|e_t|/s_t$ to ADWIN and Page–Hinkley
7. if drift flagged then
8. refit {HW, SARIMA, LightGBM}; schedule follow-ups at $t + 24, t + 72, t + 168$
9. end if
10. if a follow-up is due or 168 h have passed since the last refit then
11. refit {HW, SARIMA, LightGBM}
12. end if
13. return $\hat{y}_t$

Algorithm 2. Contextual serving agent (full-information contextual bandit)

1. initialise $\theta_j = 0$ and $P = I$ for the candidates $j \in \{c, 1, \dots, M\}$
2. for each step t do
3. build x_t (calendar, disagreement, recent losses, level change, time since alarm and refit)
4. predict the losses $\theta_j^\top x_t$; serve the softmin mix (21)
5. observe y_t; compute $\ell_{j,t}$ for every candidate (19)
6. update every θ_j and P by forgetting RLS (20)
7. end for

E. Module 5: Evaluation and Reward Engine

Module 5 scores each served forecast and returns every candidate's loss (19) to the agent. Accuracy is reported by

$$\mathrm{MAE} = 1/T \sum_{t=1}^{T} |\, y_t - \hat{y}_t|, \tag{22}$$

$$\mathrm{RMSE} = \sqrt{1/T \sum_{t=1}^{T} (y_t - \hat{y}_t)^2}, \tag{23}$$

$$\mathrm{MAPE} = 100\%/T \sum_{t=1}^{T} |y_t - \hat{y}_t / y_t|. \tag{24}$$

Equations (22)–(24) are the accuracy metrics — mean absolute error (MAE), root-mean-square error (RMSE) and mean absolute percentage error (MAPE) — computed over the T test steps, where y_t is the observed load and $\hat{y}_t$ the served forecast. RMSE is additionally reported as CV-RMSE = RMSE / $\bar{y}$ × 100, the RMSE normalised by the mean load $\bar{y}$, so that accuracy is comparable across clients of different magnitude.

Cumulative regret against the best single model in hindsight [16] is

$$\mathrm{Regret}_T = \sum_{t=1}^{T} (l(\hat{y}_t) - l(\hat{y}_t^*)). \tag{25}$$

Equation (25) is the cumulative regret against the best single model in hindsight, where $l(\hat{y}_t)$ is the loss of the served forecast and $l(\hat{y}^*_t)$ the loss of the model that is best over the same window; regret near zero means the controller tracks the best model almost as well as an oracle.

Adaptation speed is a censoring-aware recovery time: for each drift point t_0,

$$T_{\mathrm{rec}} = \min\{\tau \le 672:\ \bar{r}_{t_0+\tau+u} \le 1.25\, \bar{r}_{\mathrm{pre}},\ u = 0, \dots, 23\}, \tag{26}$$

Equation (26) defines the recovery time after a change at t_0: the smallest lag τ from which the 24-h rolling mean relative error $\bar{r} = |e_t|/y_t$ stays within 25% of its median over the two weeks before the change for 24 consecutive hours, capped at four weeks. Relative error is used so that a level shift does not count as permanent damage; events that never recover within the cap are counted separately.

Shift experiment and significance. To test adaptation under known changes, four demand shifts are injected into each client's test series at 12 and 33 weeks: level (×1.3), ramp (to ×1.3 over four weeks), daily amplitude (×1.6) and a one-week shock (×1.4). The second event reverses the first, except for the shock, which repeats. Differences between methods are tested with the Diebold–Mariano test [30] on hourly losses (Newey–West variance, 24-h lag) for every client and scenario, and with a Wilcoxon signed-rank test across the 40 resulting series.

IV. Results and Discussion

TABLE II. Test accuracy on the real data (mean over eight clients) and model refits per week.

Method	MAE	RMSE	CV-RMSE (%)	MAPE (%)	Refits/wk
Seasonal naive	135.3	206.1	13.61	9.62	0.0
Holt–Winters	107.4	151.9	9.96	9.63	1.0
SARIMA	75.2	117.3	7.92	6.25	1.0
LightGBM	78.4	125.7	7.86	5.55	1.0
Online linear	84.9	121.8	7.69	6.17	0.0
Validation-chosen single	73.5	115.2	7.47	5.19	1.0
Equal-weight ensemble	79.5	117.9	7.59	6.14	3.0
FTL	72.8	111.8	7.36	5.20	3.0
Hedge	71.0	108.7	7.11	5.13	3.0
DWM	68.7	106.5	6.96	4.98	3.0
MLpol	68.2	103.2	6.55	5.02	3.0
MELO [16]	66.5	101.4	6.47	4.82	3.0
Initial controller	69.8	106.5	6.74	5.01	3.4
SAGE (proposed)	**66.7**	**101.3**	**6.34**	**4.70**	**4.3**

A. Experimental Setup

All experiments are implemented in Python and run on a laptop CPU with no GPU: Holt–Winters and SARIMA use statsmodels, the gradient-boosted member uses LightGBM, and the online learner and both detectors use river. Every method is evaluated under the same prequential protocol on the test block (9,207 hours per client), and every setting—the drift thresholds, MELO's inflation scale, the alarm response, and the agent's λ and τ—was chosen on the validation block, including validation copies of the shift scenarios, before the test block was run once. The compared methods are the five single models; a validation-chosen single model; an equal-weight ensemble; four online aggregation baselines, Follow-

the-Leader and Hedge [29], [31], Dynamic Weighted Majority [14] and MLpol [28]; MELO [16], replayed over the same pool forecasts with weekly refits; the initial controller (top-3 inverse-error blend with a Discounted-UCB1 bandit [26], [27]); MELO with daily refits under the shift scenarios; and the proposed system, SAGE. MAPE is the primary accuracy metric, because it is scale-free and so weights small and large clients equally; CV-RMSE is reported alongside it because MAPE is inflated in hours with near-zero load.

B. Dataset Description

Experiments use the UCI ElectricityLoadDiagrams20112014 dataset, a public benchmark of individual electricity-consumption series. The raw data provide 15-minute readings for 370 clients over 2011–2015; following common practice it is aggregated to hourly resolution and restricted to the 321 clients with complete history over 2012–2014, giving 26,304 hourly observations per client. A stratified subset of eight clients is selected by ranking clients on the coefficient of variation of their consumption and sampling across the low-, medium- and high-variability terciles, so the evaluation spans both stable and volatile demand. Calendar covariates (hour-of-day, day-of-week, month, holiday flag) are attached to every timestamp and each series is causally standardised as in (2).

C. Accuracy on the Real Data

Table II and Fig. 3 compare all methods on the real test data. SAGE has the lowest MAPE (4.70%) and the lowest CV-RMSE (6.34%) of every method: MELO reaches 4.82% and 6.47%, the best other online aggregation method, DWM, 4.98% MAPE, the best single model chosen on validation 5.19%, and the equal-weight ensemble 6.14%. Its MAE is slightly above MELO's, because the agent leans on the individual models in low-load hours, where percentage errors are largest. Every online aggregation method beats the initial top-3 blend, which is why MELO replaced it as the combiner.

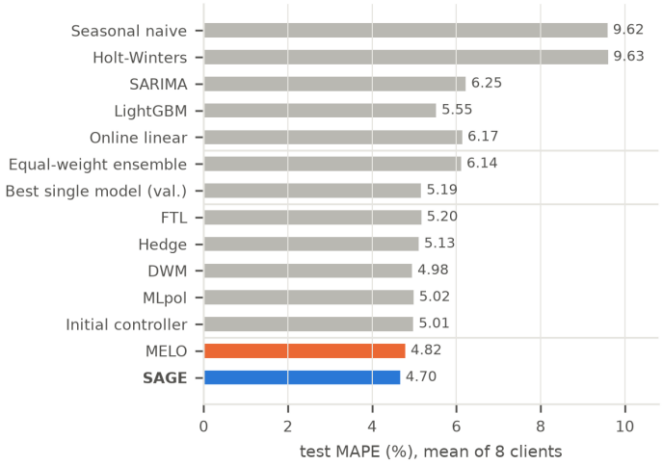


Fig. 3. Test MAPE by method (mean of eight clients); blue: SAGE, orange: MELO.

Fig. 4 compares SAGE with the four strongest baselines client by client. SAGE has lower CV-RMSE than Hedge, DWM and MLpol on every client and lower MAPE on seven of the eight (on MT_269 DWM is 0.001 point ahead); it beats MELO on MAPE on 4 of the eight. On the four most regular clients it is within 0.1 point of MELO, with MELO marginally ahead; its large gains come on the two irregular clients examined in Section IV-E.

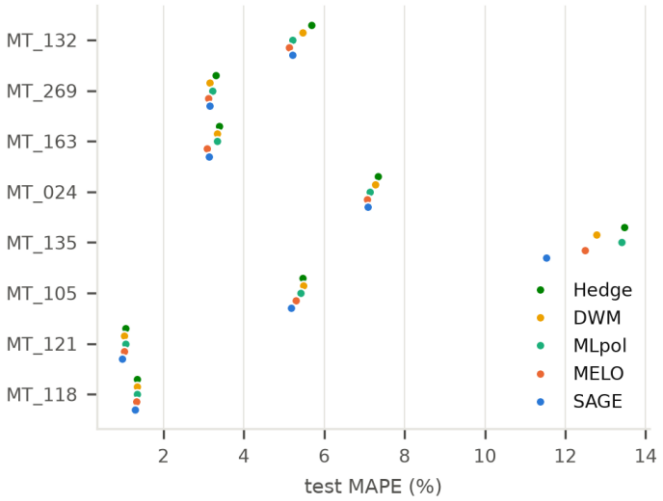


Fig. 4. Per-client test MAPE for SAGE and the four strongest baselines: Hedge, DWM, MLpol and MELO.

D. Adaptation Under Injected Demand Shifts

Table III and Fig. 5 report the shift experiment. SAGE has lower MAPE and lower CV-RMSE than MELO in every scenario, while Hedge, DWM and MLpol are worse than MELO in every scenario. SAGE also has lower MAPE than MELO refitted every day, in every scenario, while refitting about five times less often; on CV-RMSE the two are level. The detector caught 10 of 16 level shifts and 14 of 16 temporary shocks (median delays 25 h and 30 h) but only 5 of 16 slow ramps and 9 of 16 amplitude changes, with about 0.2 false alarms per week. After temporary shocks SAGE recovered in a median 133 h against 149 h for MELO, but after level shifts it was slower (138 h against 114 h). In the amplitude scenario MT_135's near-zero hours inflate MAPE, so CV-RMSE is the fairer measure there.

TABLE III. Injected demand shifts: test MAPE / CV-RMSE (%), mean over eight clients, and shift detection by SAGE (events caught of 16, median detection delay).

Method	No shift	Level	Ramp	Amplitude	Temporary
MELO, weekly refit	4.82 / 6.47	4.90 / 6.61	4.86 / 6.50	8.70 / 7.68	4.94 / 6.80
MELO, daily refit	–	4.84 / 6.48	4.80 / 6.39	8.95 / 7.59	4.88 / 6.57
SAGE (proposed)	**4.70 / 6.34**	**4.75 / 6.45**	**4.74 / 6.40**	**8.38 / 7.59**	**4.79 / 6.58**
Caught, median delay	–	10/16, 25 h	5/16, 157 h	9/16, 81 h	14/16, 30 h

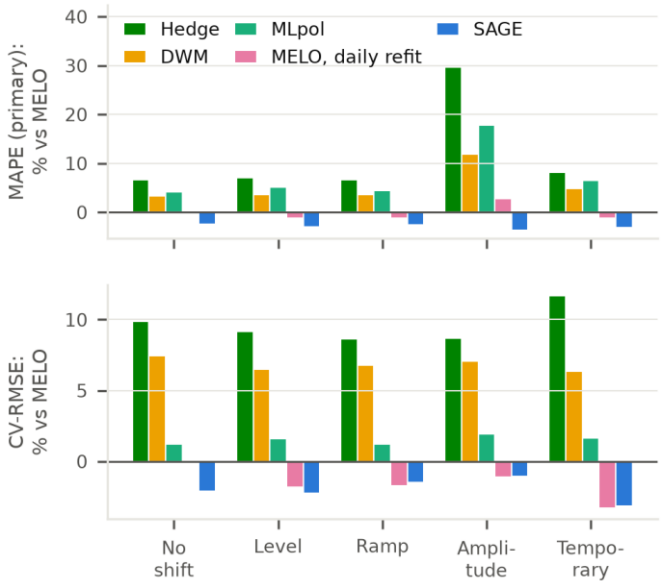


Fig. 5. Change in test MAPE (top) and CV-RMSE (bottom) relative to MELO for Hedge, DWM, MLpol, MELO with daily refits and SAGE; below zero is better.

Fig. 6 follows one event, chosen as the median, by MAPE change against MELO over the next two weeks, of the ten level shifts the detector caught. Load on MT_132 drops back by the 30% added earlier; the detector fires after 17 h, the models are refit then and at the follow-up times, and the agent briefly shifts weight from MELO toward SARIMA and the online model while the pool re-adapts. Across all sixteen level-shift events SAGE beats MELO over the following two weeks in eight and loses in eight; most of its gain comes from normal operation and from the cases below.

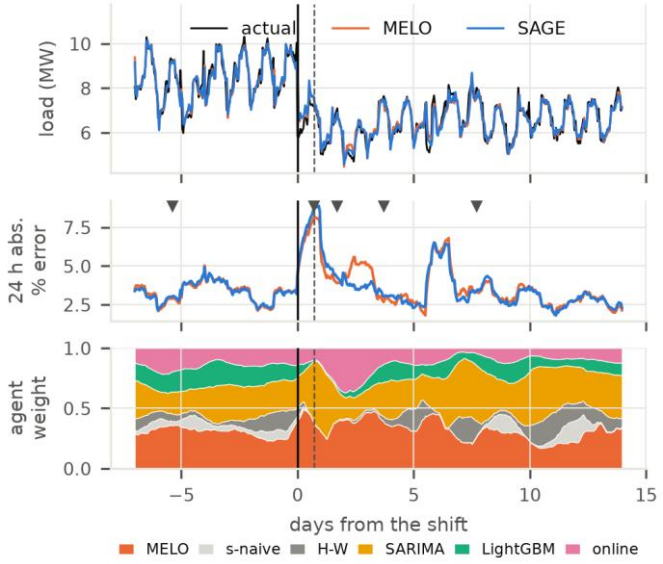


Fig. 6. Injected level shift on MT_132: load and forecasts (top), 24-h rolling absolute % error with refits ▼ (middle), agent weights (bottom). Solid line: shift; dashed: alarm.

E. Where SAGE Helps Most

The largest gains over MELO come from two real clients. MT_105 (Fig. 7) is the largest CV-RMSE gain (5.79% vs 6.93%). After four quiet weeks at about 100 kW, its load jumped about fivefold on 23 October 2014. SARIMA tracked the new level, and the agent moved almost all of its weight onto SARIMA within five hours; the detector fired 7 h and 15 h after the jump, and the refits it triggered included the new data. In the weekly-refit pool used by MELO, SARIMA fell back to the old level nine hours after the jump, and MELO's correction experts, which can extrapolate beyond the pool, overshot: at that hour it forecast 1,993 kW against an actual 824 kW. That single hour accounts for 84% of the difference in squared error on this client; without it SAGE is still better (5.80% vs 5.99%).

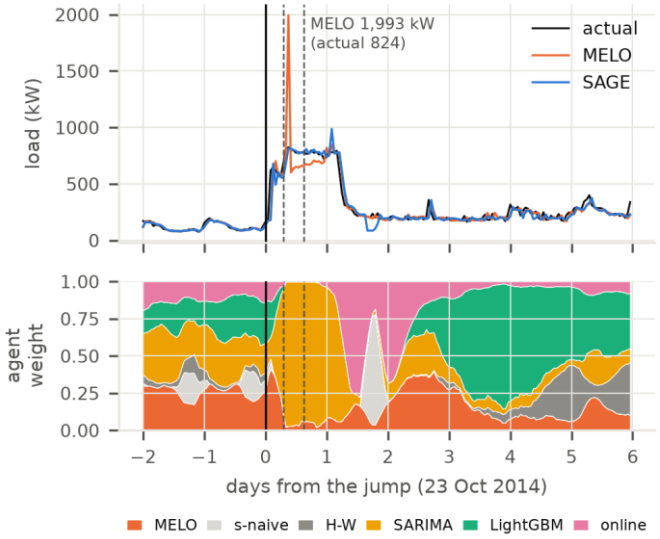


Fig. 7. MT_105, real data: a fivefold load jump on 23 October 2014. Load and forecasts (top) and SAGE's agent weights (bottom, 6-h mean). Solid line: the jump; dashed: drift alarms.

MT_135 (Fig. 8) is the largest MAPE gain (11.53% vs 12.48%, and 7–9% lower in every shift scenario). Its load profile is inverted: it peaks at night at about 680 kW and falls to 150–300 kW by day. In the quietest quarter of hours MELO over-forecasts by 13 kW on average, which is a large percentage of a small load; the agent learns that in these hours the seasonal-naive forecast (the same hour one week earlier) is more reliable and raises its weight there from 0.16 to 0.30, and cuts the bias to 9 kW and the MAPE of those hours from 17.7% to 14.1%. The other large gains are at the morning and evening transitions (08:00, 19:00–21:00), where it leans on LightGBM. This client supplies most of SAGE's mean MAPE gain over MELO.

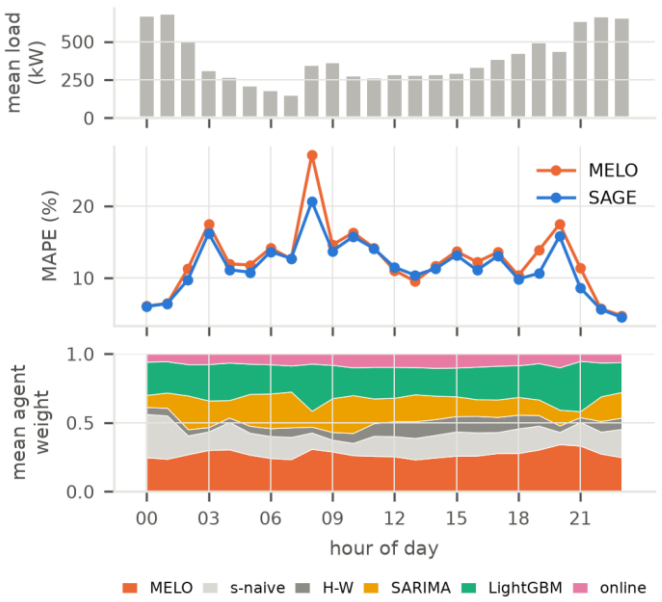


Fig. 8. MT_135, real test data, by hour of day: mean load (top), MAPE of MELO and SAGE (middle), and SAGE's mean agent weights (bottom).

F. Statistical Significance and Cost

Table IV summarises the Diebold–Mariano tests of SAGE against MELO. SAGE is significantly better overall on absolute, squared and percentage error (Wilcoxon p = 0.01–0.04 across the 40 series) and significantly better in 17 series on absolute and percentage error, but significantly worse in 5–8. In an ablation without the contextual agent, the drift response alone gives the same CV-RMSE gain and about 0.1 point higher MAPE; the agent's own contribution is not significant (p = 0.25–0.33).

TABLE IV. Diebold–Mariano tests of SAGE against MELO over 40 series (8 clients × 5 scenarios): series favouring SAGE, series significant at 5% each way, and Wilcoxon signed-rank p.

Comparison	Loss	Better in	Sig. better	Sig. worse	Wilcoxon p
SAGE vs MELO	abs.	21/40	17	5	0.013
SAGE vs MELO	sq.	22/40	3	1	0.037
SAGE vs MELO	% err.	20/40	17	8	0.025

SAGE refits its batch models about 4.3 times per week, against 3.0 for MELO with weekly refits and 21 with daily refits, and runs at about 31 ms per hourly step on a laptop CPU, of which the agentic logic—drift detection, MELO and the agent—is 1.9%.

G. What Did Not Work

Several designs were tested on validation data and rejected: the initial top-3 inverse-error blend, which every online aggregation method beat; refitting on post-change data only after an alarm; resetting MELO's regrets and covariances on alarms; and training the agent on squared rather than absolute loss. The evaluation also exposed four faults in the first implementation—a seasonal-phase error in Holt–Winters, SARIMA forecasting up to 168 h ahead without new observations, a drift detector that fired on a fixed weekly timer, and a recovery metric that always returned one hour—which inflated the earlier error figures and were corrected before every result reported here.

V. Conclusion

This work built SAGE, an hourly, GPU-free adaptive forecasting system for client-level electricity load: a pool of five inexpensive forecasters combined by MELO, an explicit drift detector whose alarms trigger immediate and follow-up refits, and a contextual agent that re-weights the combiner and the models each hour. On eight UCI clients it has the lowest MAPE and CV-RMSE of all methods tested and is significantly more accurate than MELO, the strongest online aggregation baseline; under injected demand shifts it beats daily refitting on MAPE and matches it on CV-RMSE at a fifth of the retraining cost. Its largest gains come from sudden real level changes and irregular load profiles; on regular clients it matches MELO. Richer context such as weather and calendar events, and giving the agent control over refit timing, are natural next steps.

References

[1] M. Cheng, X. Tao, Q. Liu, Z. Guo, and E. Chen, "Position: Beyond Model-Centric Prediction – Agentic Time Series Forecasting," arXiv preprint, 2026.
[2] C. Feng and J. Zhang, "Reinforcement Learning based Dynamic Model Selection for Short-Term Load Forecasting," in Proc. IEEE PES General Meeting (PESGM), 2019.
[3] Wu, Ma, and Aliev, "A Novel Reinforcement Learning-Based Approach for Short-Term Load and Price Forecasting in Energy Markets," Scientific Reports, 2026.
[4] Y. Fu, D. Wu, and B. Boulet, "Reinforcement Learning Based Dynamic Model Combination for Time Series Forecasting," in Proc. AAAI Conf. on Artificial Intelligence, 2022.
[5] S. S. S. Das et al., "Nexus: An Agentic Framework for Time Series Forecasting," arXiv preprint, 2026.
[6] Y. Zuo, D. Qin, and Y. Wang, "Large Language Model-Empowered Interactive Load Forecasting," arXiv preprint, 2025.
[7] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A Survey on Concept Drift Adaptation," ACM Computing Surveys, vol. 46, no. 4, 2014.
[8] R. Jagait et al., "Load Forecasting Under Concept Drift: Online Ensemble Learning with Recurrent Neural Network and ARIMA," IEEE Access, 2021.
[9] Y. Zhang et al., "Addressing Concept Shift in Online Time Series Forecasting: Detect-then-Adapt," arXiv preprint, 2024.
[10] Q. Zhao and Z. Shen, "Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting," in Proc. ACM SIGKDD, 2024.
[11] Lu et al., "Autonomous Concept Drift Threshold Determination," in Proc. AAAI Conf. on Artificial Intelligence, 2026.
[12] C. Hu, Y. Ma, Y. Wu, and Y. Hou, "Resilient Load Forecasting under Climate Change: Adaptive Conditional Neural Processes for Few-Shot Extreme Load Forecasting," in Proc. Int. Conf. on Machine Learning (ICML), 2026.
[13] J. Zhu, S. Liu, D. Weng, and Y. Wu, "Dynamic TMoE: A Drift-Aware Dynamic Mixture-of-Experts Framework for Non-Stationary Time Series Forecasting," in Proc. Int. Conf. on Machine Learning (ICML), 2026.
[14] J. Z. Kolter and M. A. Maloof, "Dynamic Weighted Majority: An Ensemble Method for Drifting Concepts," Journal of Machine Learning Research, vol. 8, 2007.
[15] Y. Zhang et al., "OneNet: Enhancing Time Series Forecasting Models under Concept Drift by Online Ensembling," in Proc. NeurIPS, 2023.
[16] Y. Wang, Y. Goude, and Q. Yao, "Hedging Memory Horizons for Non-Stationary Prediction via Online Aggregation," arXiv preprint, 2026.

[17] Y. He, F. Uhrweiller, and B. Sick, "Towards Continuous Power Forecasting: Practical Continual Learning for Real-World Energy Systems in Nonstationary Time Series," arXiv preprint, 2026.
[18] N. Karaouli, D. Coquenet, E. Fromont, M. Mermillod, and M. Reyboz, "Foundation vs. Specialized Models: Evaluating Catastrophic Forgetting in Continual Time Series Forecasting," arXiv preprint, 2025.
[19] Z. Shao et al., "Exploring Progress in Multivariate Time Series Forecasting: Comprehensive Benchmarking and Heterogeneity Analysis," IEEE Trans. on Knowledge and Data Engineering, 2024.
[20] T. Hong et al., "Energy Forecasting: A Review and Outlook," IEEE Open Access Journal of Power and Energy, vol. 7, 2020.
[21] P. R. Winters, "Forecasting Sales by Exponentially Weighted Moving Averages," Management Science, vol. 6, no. 3, 1960.
[22] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. Holden-Day, 1970.
[23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Proc. NeurIPS, 2017.
[24] A. Bifet and R. Gavalda, "Learning from Time-Changing Data with Adaptive Windowing," in Proc. SIAM Int. Conf. on Data Mining (SDM), 2007.
[25] E. S. Page, "Continuous Inspection Schemes," Biometrika, vol. 41, no. 1/2, 1954.
[26] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," Machine Learning, vol. 47, 2002.
[27] A. Garivier and E. Moulines, "On Upper-Confidence Bound Policies for Switching Bandit Problems," in Proc. Algorithmic Learning Theory (ALT), 2011.
[28] P. Gaillard, G. Stoltz, and T. van Erven, “A Second-order Bound with Excess Losses,” in Proc. Conf. on Learning Theory (COLT), 2014.
[29] N. Cesa-Bianchi and G. Lugosi, Prediction, Learning, and Games. Cambridge University Press, 2006.
[30] F. X. Diebold and R. S. Mariano, “Comparing Predictive Accuracy,” Journal of Business & Economic Statistics, vol. 13, no. 3, 1995.
[31] N. Littlestone and M. K. Warmuth, “The Weighted Majority Algorithm,” Information and Computation, vol. 108, no. 2, 1994.